Sinhgad Technical Education Society's

SINHGAD COLLEGE OF ENGINEERING, Vadgaon 411041

NAAC Accredited with 'A' Grade

Sinhgad Institutes

LABORATORY MANUAL

INFORATION STORAGE AND RETRIEVAL LAB [414446]

414446: Lab Practice III

Final Year of Information Technology
(2019 Course)

A.Y. 2026- 2027

Semester – I

DEPARTMENT OF INFORMATION TECHNOLOGY

Vision and Mission of Institute

VISION

उत्तमपुरुषान् उत्तमाभियंतृन् निर्मातुं कटीबध्दाः वयम्।

MISSION

- We believe in and work for the holistic development of students and teachers.
- We strive to achieve this by imbibing a unique value system, transparent work culture, excellent academic and physical environment conducive to learning, creativity and technology transfer.

Vision and Mission of Department

VISION

The department of Information Technology is committed to grow on a path of delivering distinctive high quality education, fostering research, creativity and innovation.

MISSION

- The department of Information Technology in partnership with all stake holders will harness Talent, Potential for application based indigenous product development in future.
- Our Endeavour is to provide conductive environment for life skill development of students while exercising effective Learning Strategies.

Program Educational Objectives (PEO's):

I. Graduates of the programme will be prepared to work as an IT professional.

II. IT Graduates will function effectively as individuals and team members, growing into technical and leadership roles.

III. Graduates of the programme will pursue continuous learning required to adapt and flourish in ever changing scenarios to pursue career in IT/non IT professions.

Program Outcomes (PO's):

POs are statements that describe what students are expected to know and be able to do upon graduating from the program. These relate to the skills, knowledge, analytical ability attitude and behavior that students acquire through the program.

a) Engineering knowledge: Graduates will be able to apply the Knowledge of the mathematics, science and engineering fundamentals for the solution of engineering problems related to IT.

b) Problem analysis: Graduates will be able to carry out identification and formulation of the problem statement by requirement engineering and literature survey.

c) Design/development of solutions: Graduates will be able to design a system, its components and/or processes to meet the required needs with consideration for public safety and social considerations.

d) Conduct investigations of complex problems: Graduates will be able to investigate the problems, categorize the problem according to their complexity using modern computational concepts and tools.

e) Modern tool usage: Graduates will be able to use the techniques, skills, modern IT engineering tools necessary for engineering practice.

f) The engineer and society: Graduates will be able to apply reasoning and knowledge to assess global and societal issues

g) Environment and sustainability: Graduates will be able to recognize the implications of engineering IT solution with respect to society and environment.

h) Ethics: Graduates will be able to understand the professional and ethical responsibility.

i) Individual and team work: Graduates will be able to function effectively as an individual member, team member or leader in multi-disciplinary teams.
j) Communication: Graduates will be able to communicate effectively and make effective documentations and presentations.

k) Project Management and Finance: Graduates will be able to apply and demonstrate engineering and management principles in project management as a member or leader.

l) Life-long Learning: Graduates will be able to recognize the need for continuous learning and to engage in life-long learning.

Program Specific Outcomes (PSO)

1. An ability to employ technical concepts and practices in information technologies - Software Engineering, information management, programming, networking and communications, web technologies

2. An ability to understand the computational fundamentals and computing resources

3. An ability to use systems for securely processing, storing, retrieving and transmitting information

Course Objectives and Course Outcomes (COs)

Course Objectives:

1. To understand information retrieval process.
2. To understand concepts of clustering and how it is related to Information retrieval.
3. To deal with Storage, Organization & Access to Information Items.
4. To evaluate the performance of IR system and understand user interfaces for searching.
5. To understand information sharing on semantic web.
6. To understand the various applications of Information Retrieval giving emphasis to multimedia and distributed IR, web Search.
7. To apply the gained knowledge in recent fields of advancements in the subject.

Course Outcomes:

By the end of the course, students should be able to,

1. Understand the concept, data structure and preprocessing algorithms of Information retrieval.
2. Deal with storage and retrieval process of text and multimedia data.
3. Evaluate performance of any information retrieval system.
4. Design user interfaces.
5. Understand importance of recommender system (Take decision on design parameters of recommender system).
6. Understand concept of multimedia and distributed information retrieval.
7. Map the concepts of the subject on recent developments in the Information retrieval field.

Program Specific Outcomes: PSOs

1. Get solid foundation in design and development of software application useful to society
2. Able to developed Programming skills

Sinhgad Technical Education Society's

SINHGAD COLLEGE OF ENGINEERING, VADGAON(BK)
NAAC Accredited with "A" Grade



CERTIFICATE

This is to certify that Mr. /Ms. _____

of class BE IT Div_____Roll No._____Examination Seat No./PRN No. _____

has completed all the practical work in the Information Storage and Retrieval Lab [**414446**] satisfactorily, as prescribed by Savitribai Phule Pune University, Pune in the academic year 2022-23 (Sem I)

Place:

Date:

Course In-charge Head of Department Principal

INDEX

Sr. No	Title of Experiment	Date of performance	Date of Submission	Marks Obtained (10)	Signature of Faculty
1	Implement Conflation algorithm to generate document representative of a text file.				
2	Implement Single-pass Algorithm for clustering of files.(Consider 4 to 5 files)				
3	Implement a program for retrieval of documents using inverted files.				
4	Implement a program to calculate precision and recall for sample input. (Answer set A, Query q1, Relevant documents to query q1-Rq1)				
5	Write a program to calculate harmonic mean (F-measure) and E-measure for above example.				
6	Implement a program for feature extraction in 2D color images (any features like color, texture etc. and to extract features from input image and plot histogram for the features.				
7	Build the web crawler to pull product information and links from an e-commerce website. (Python)				
8	Write a program to find the live weather report (temperature, wind speed, description, and weather) of a given city. (Python).				
9	Case study on recommender system for a product / Doctor / Product price / Music.				

Name & Signature of Course In-charge

LAB INNOVATION

Note: Student are expected to do minimum one lab innovation

1. Implement of recommender system
2. Graphical representing the output of clustering algorithm
3. Web page rank calculation for some scenario
4. Calculation of precision and recall for some set of documents and queries

Aim:1) Implement Conflation algorithm to generate document representative of a text file.

Objectives:
To study Conflation Algorithm & Document Representative
Outcomes:

At the end of the assignment the students should have
1. Understood the design and implementation of conflation algorithm

PEOs, POs, PSOs and COs satisfied

POs: a,b,c,d,e **PEOs: I,III** **PSOs: 1,3** **COs: 1,2**

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS, C/C++/Java

Theory:

Document Representative:
Documents in a collection are frequently represented through a set of index terms or keywords. Such keywords might be extracted directly from the text of the document or might be specified by a human subject. Modern computers are making it possible to represent a document by its full set of words. With very large collections, however, even modern computers might have to reduce the set of representative keywords. This can be accomplished through the elimination of *stop words* (such as articles and connectives), the use of *stemming* (which reduces distinct words to

their common grammatical root), and the identification of noun groups (which eliminates adjectives, adverbs, and verbs). Further, compression might be employed. These operations are called *text operations* (or transformations). The full text is clearly the most complete logical view of a document but its usage usually implies higher computational costs. A small set of categories (generated by a human specialist) provides the most concise logical view of a document but its usage might lead to retrieval of poor quality. Several intermediate logical views (of a document) might be adopted by an information retrieval system as illustrated in Figure

Besides adopting any of the intermediate representations, the retrieval system might also recognize the internal structure normally present in a document. This information on the structure of the document might be quite useful and is required by structured text retrieval models. As illustrated in Figure we view the issue of logically representing a document as a continuum in which the logical view of a document might shift (smoothly) from a full text representation to a higher-level representation specified by a human subject.

The document representative is one consisting simply of a list of class names, each name representing a class of words occurring in the total input text. A document will be indexed by a name if one of its significant words occurs as a member of that class.

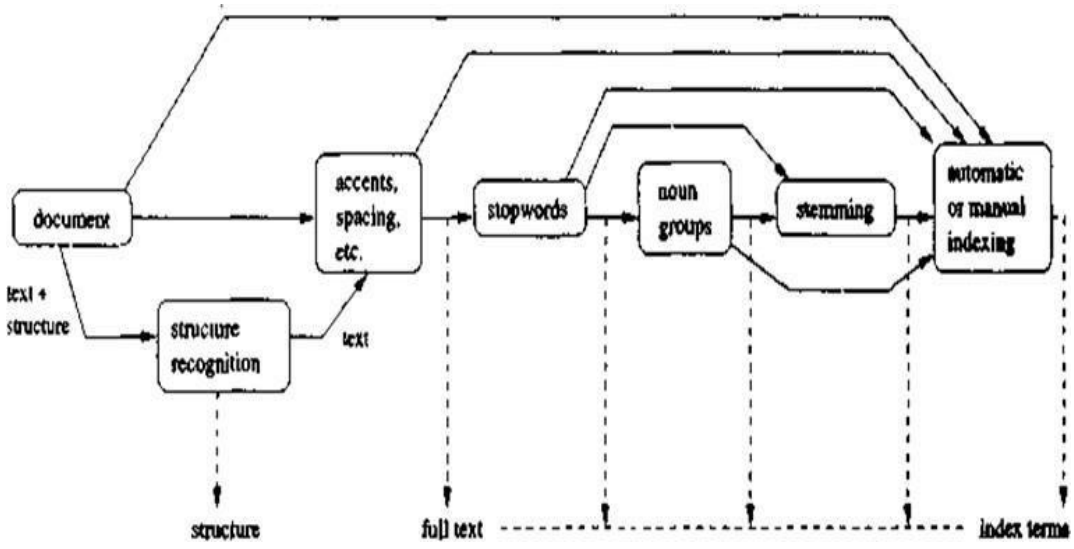


Figure: Logical view of document: full text to set of index terms

Conflation Algorithm: Steps for Conflation Algorithm

1. Removal of high frequency words
2. Suffix stripping
3. Detecting equivalent stems.

1. Removal of high frequency words:

The removal of high frequency words, 'stop' words or 'fluff' words. This is normally done by comparing the input text with a 'stop list' of words which are to be removed. The advantages of the process are not only that non-significant words are removed and will therefore not interfere during retrieval, but also that the size of the total document file can be reduced by between 30 and 50 per cent.

2. Suffix Stripping:

The second stage, suffix stripping, is more complicated. A standard approach is to have a complete list of suffixes and to remove the longest possible one. For example, we may well want UAL removed from FACTUAL but not from EQUAL. To avoid erroneously removing suffixes, context rules are devised so that a suffix will be removed only if the context is right. 'Right' may mean a number of things:

(1) the length of remaining stem exceeds a given number; the default is usually 2;

(2) the stem-ending satisfies a certain condition, e.g. does not end with Q.

3. Detecting equivalent stems:

Many words, which are equivalent in the above sense, map to one morphological form by removing their suffixes. The simplest method of dealing with it is to construct a list of equivalent stem-endings. For two stems to be equivalent they must match except for their endings, which themselves must appear in the list as equivalent. For example, stems such as ABSORB- and ABSORPT- are conflated because there is an entry in the list defining B and PT as equivalent stem-endings if the preceding characters match.

The assumption (in the context of IR) is that if two words have the same underlying stem then they refer to the same concept and should be indexed as such. This is obviously a very simplification since words with the same stem, such as NEUTRON AND NEUTRALISE, sometimes need to be distinguished. Even words which are essentially equivalent may mean different things in different contexts. Since there is no cheap way of making these fine

distinctions we put up with a certain proportion of errors and assume (correctly) that they will not degrade retrieval effectiveness too much.

The final output from a conflation algorithm is a set of classes, one for each stem detected. A class name is assigned to a document if and only if one of its members occurs as a significant word in the text of the document. A document representative then becomes a list of class names. These are often referred to as the documents *index terms* or *keywords*.

Conclusion: Thus we have implemented conflation algorithm

A. Write short answer of following questions:

1. Explain the working of conflation algorithm.
2. State and explain Luhn's theroy

B. Viva Questions:

1. Difference between Data Retrieval and Information

2. Indexing exhaustivity and specificity.

3. Five commonly used measures of association in information retrieval.

4. Why Normalized versions of the simple matching coefficient are used for measures of association

Aim:2) Implement Single-pass Algorithm for clustering of files.(Consider 4 to 5 files)

Objectives:

To study Clustering using single pass algorithm

Outcomes:

At the end of the assignment the students should have

1. Understood the working and implementation of single pass clustering algorithm

PEOs, POs, PSOs and COs satisfied

POs: a,b,c,d **PEOs: I,III** **PSOs: 1,3** **Cos: 1,2**

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS

Theory: -

Clustering: A basic assumption in retrieval systems is that documents relevant to a request are separated from those which are not relevant, i.e. that the relevant documents are more like one another than they are like non-relevant documents. Whether this is true for a collection can be tested as follows. Compute the association between all pairs of documents:

(a) Both of which are relevant to a request, and

(b) One of which is relevant and the other non-relevant.

In choosing a cluster method for use in experimental IR, two criteria have frequently been used. The first of these is the *theoretical soundness* of the method. It means that the method should satisfy certain criteria of adequacy. To list some of the more important of these:

(1) The method produces a clustering which is unlikely to be altered drastically when further objects are incorporated, i.e. it is stable under growth.

(2) The method is stable in the sense that small errors in the description of the objects lead to small changes in the clustering;

(3) The method is independent of the initial ordering of the objects.

The second criterion for choice is the *efficiency* of the clustering process in terms of speed and storage requirements.

The two distinct approaches to clustering can be identified:

(1) The clustering is based on a measure of similarity between the objects to be clustered;

(2) The cluster method proceeds directly from the object descriptions.

The most obvious examples of the first approach are the *graph theoretic* methods which define clusters in terms of a graph derived from the measure of similarity. The example of second approach is Single Pass algorithm.

The most important concept is that of *cluster representative* variously called cluster profile, classification vector, or centroid. It is simply an object which summaries and represents the objects in the cluster. Ideally it should be near to every object in the cluster in some average sense; hence the use of the term centroid. The similarity of the objects to the representative is measured by a *matching function*.

The algorithms also use a number of *empirically* determined parameters such as:

(1) The number of clusters desired;

(2) A minimum and maximum size for each cluster;

(3) A threshold value on the matching function, below which an object will not be included in a cluster;

(4) The control of overlap between clusters;

(5) An arbitrarily chosen objective function which is optimized.

Cluster Hypothesis: closely associated documents tend to be relevant to the same requests.

Single Pass Algorithm:

1. The object descriptions are processed serially;
2. The first object becomes the cluster representative of the first cluster;
3. Each subsequent object is matched against all cluster representatives existing at its processing time;
4. A given object is assigned to one cluster (or more if overlap is allowed) according to some condition on the matching function;
5. When an object is assigned to a cluster the representative for that cluster is recomputed;
6. If an object fails a certain test it becomes the cluster representative of a new cluster.

Algorithm:

1. Input minimum five conflated files
2. Define cluster one and initialize the first object as object descriptive for that
3. Calculate dice coefficient between first object which is object descriptive and next object
4. If matching coefficient is greater than threshold, then add to defined cluster else create new cluster with object descriptive as that object.
5. Process next object serially.

Print the cluster number with objects

Conclusion: Implementation is concluded by stating analysis of single pass algorithm for clustering with the benefits and limitations.

A. Write short answer of following questions :

1. Explain Clustering using dissimilarity matrix. Also explain effect of threshold on clustering.
2. Explain K-list
3. Explain Cluster-based retrieval
4. Working of Rochio's algorithm

B. Viva Questions:

1. Cluster using similarity measures
2. IR Models
3. Boolean Search
4. What is multipass clustering technique.

Aim :3) Implement a program for retrieval of documents using inverted files.Objective: -
To study Indexing, Inverted Files and searching information with the help of inverted file

Outcomes:

At the end of the assignment the students should have

1. Understood use of indexing in fast retrieval
2. Understood working of inverted index

PEOs, POs, PSOs and COs satisfied :

POs: a,b,c,d PEOs: I,III PSOs: 1,3 Cos: 1,2

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS/C/C++/Java

Theory:

Indexing

In searching for a basics query is to scan the text sequentially. Sequential or online text searching involves finding the occurrences of a pattern in a text. Online searching is appropriate then the text is small and it is the only choice if the text collection is very volatile or the index space overhead cannot be afforded. A second option is to build data

structures over the text to speed up the search. It is worthwhile building and maintaining an. index when the text collection is large and semi-static. Semi-static collections can be updated at reasonably regular intervals but they are not deemed to support thousands of insertions of single words per second. This is the case for most real text databases not only dictionaries or other slow growing literary works. There are many indexing Techniques. Three of them are inverted files, suffix arrays and signature files.

Inverted Files:

An inverted file is a word-oriented mechanism for indexing a test collection in order to speed up the matching task. The inverted file structure is composed of two elements: vocabulary and occurrence. The vocabulary is the set of all different words in the text. For each such word a list of all the text portions where the word appears is stored. The setof all those lists is called the occurrences. These positions can refer to words or characters. Word

positions simplify phrase and proximity queries, while character positions facilitate direct access to the matching text position.

Searching with the help of inverted file:

The search algorithm on an inverted index has three steps.

1. Vocabulary Search

Searching with the help of inverted file:

The search algorithm on an inverted index has three steps.

1. Vocabulary Search
2. Retrieval of occurrence
3. Manipulation of occurrences

Single-word queries can be searched using any suitable data structure to speed up the search, such as hashing, tries, or B-trees. The first two give *O(m)* search cost. However, simply storing the words in lexicographically order is cheaper in space and very competitive in performance. Since the word can be binary searched at *O (log n)* cost. Prefix and range queries can also be solved with binary search, tries, or B-trees but not with hashing. If the query is formed by single words then the process ends by delivering the list of occurrences. Context queries arc more difficult to solve with inverted indices. Each element must be searched separately and a list

generated for each one. Then, the lists of all elements are traversed in synchronization to *find* places where all the words appear in sequence (for a phrase) or appear close enough (for proximity). If one list is much shorter than the others, it may be better to binary search its elements into the longer lists instead of performing a linear merge. If block addressing is used itis necessary to traverse the blocks for these queries, since the position information is needed. It isthen better to intersect the lists to obtain the blocks which contain all the searched words andthen sequentially search the context query in those blocks. Some care has to be exercised at block boundaries, since they can split a match.

Example:

Text:

1 6 9 11 17 19 ….
This is a text. A text has many words. Words are made from letters.

Inverted Index:

Vocabulary	Occurrences
Letters	60…
Made	50…
Many	28…
Text	11,19…
Words	33,40….

Algorithm
1. Input the conflated file 2. Build the index file for input file 3. Input the query 4. Print the index file and result of query

Conclusion: Implementation is concluded by stating analysis of Retrieval of documents using Inverted Files.

A. Write short answer of following questions:

1. Working of inverted files.
2. What are applications of inverted index.
3. Working of signature files

B. Viva Questions:

1. What is vocabulary and occurrences

2. How search is carried out on inverted index

3. How to index multimedia object.

4. Limitations of inverted index.

5. What is Suffix Array and Suffix Tree.

6. What is the concept of signature files.

Aim **:** Implement a program to calculate precision and recall for sample input.

Objective:

To study how to calculate precision and recall for sample input

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS, C/C++/.Java

Theory:

Introduction:

Precision and Recall in Information Retrieval

Information Systems can be measured with two metrics: precision and recall. When a user decides to search for information on a topic, the total database and the results to be obtained can be divided into 4 categories:

1. Relevant and Retrieved
2. Relevant and Not Retrieved
3. Non-Relevant and Retrieved
4. Non-Relevant and Not Retrieved

Relevant items are those documents that help the user in answering his question. Non-Relevant items are items that don't provide actually useful information. For each item there are two possibilities it can be retrieved or not retrieved by the user's query. Precision is defined as the ratio of the number of relevant and retrieved documents (number of items retrieved that are

actually useful to the user and match his search need) to the number of total retrieved documents from the query. Recall is defined as ratio of the number of retrieved and relevant documents(the

number of items retrieved that are relevant to the user and match his needs) to the number of possible relevant documents(number of relevant documents in the database).Precision measures one aspect of information retrieval overhead for a user associated with a particular search. If a search has 85 percent precision then 15(100-85) percent of user effort is overhead reviewing non-relevant items. Recall measures to what extent a system processing a particular query is able to retrieve the relevant items the user is interested in seeing. Recall is a very useful concept but due to the denominator is non-calculable in operational systems. If the system is made known the total set of relevant items in the database, recall can be made calculable.

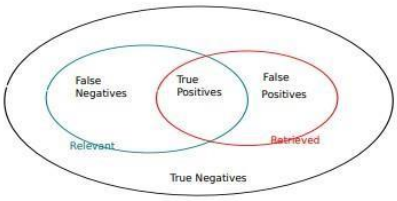
- Precision (P) is the fraction of retrieved documents that are relevant

$$\text{Precision} = \frac{\#(\text{relevant items retrieved})}{\#(\text{retrieved items})} = P(\text{relevant}|\text{retrieved})$$

- Recall (R) is the fraction of relevant documents that are retrieved

$$\text{Recall} = \frac{\#(\text{relevant items retrieved})}{\#(\text{relevant items})} = P(\text{retrieved}|\text{relevant})$$

		THE TRUTH	
WHAT THE SYSTEM THINKS		Relevant	Nonrelevant
	Retrieved	true positives (TP)	false positives (FP)
	Not retrieved	false negatives (FN)	true negatives (TN)



$$P = TP/(TP + FP)$$
$$R = TP/(TP + FN)$$

Precision/recall trade-off

You can increase recall by returning more docs. Recall is a non-decreasing function of the number of docs retrieved. A system that returns all docs has 100% recall! The converse is also true (usually): It's easy to get high precision for very low recall.

Consider an Information retrieval (IR) system returning relevant documents

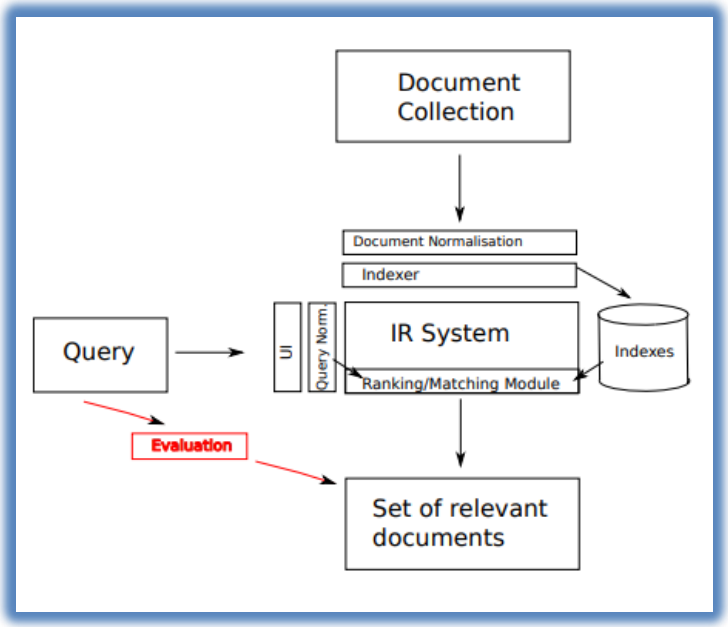


Fig 1: IR system returning relevant documents

Precision and Recall explanation:

Consider,

I: an information request

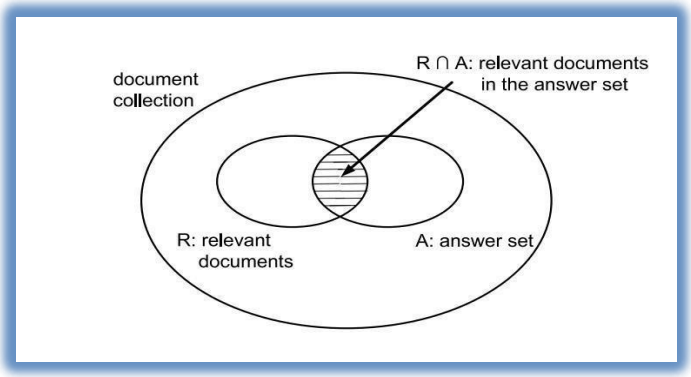
R: the set of relevant documents for I

A: the answer set for I, generated by an IR system

R ∩ A: the intersection of the sets R and A

|A|-number of documents in the set A

|Ra |-number of documents in the intersection of sets R and A

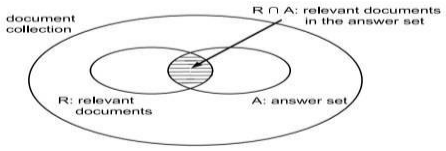


- **Recall** is the fraction of the relevant documents (the set R) which has been retrieved i.e.,

$$Recall = \frac{|R \cap A|}{|R|}$$

- **Precision** is the fraction of the retrieved documents (the set A) which is relevant i.e.,

$$Precision = \frac{|R \cap A|}{|A|}$$



Recall

$$R = \frac{\text{Number of relevant items retrieved}}{\text{Total number of relevant items in collection}}$$

Precision

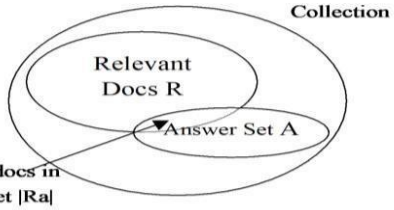
$$P = \frac{\text{Number of relevant items retrieved}}{\text{Total number of items retrieved}}$$

The goal is to achieve high precision and high recall. The definition of precision and recall assumes that all docs in the set A have been examined However, the user is not usually presented with all docs in the answer set A at once User sees a ranked set of documents and examines them starting from the top Thus, precision and recall vary as the user proceeds with their examination of the set A. Most appropriate then is to plot a curve of precision versus recall.

I	Information request
R	set of relevant documents
|R|	number of documents in set R
A	Document answer set
|A|	Number of documents in the set A
$\|R_a\|$	Number of documents in the intersection of sets R and A

$$\text{Recall} = \frac{|\,Ra\,|}{|\,R\,|}$$

$$\text{Precision} = \frac{|\,Ra\,|}{|\,A\,|}$$



- Consider a reference collection and a set of test queries
- Let R_{q_1} be the set of relevant docs for a query q_1:
  - $R_{q_1} = \{d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{89}, d_{123}\}$
- Consider a new IR algorithm that yields the following answer to q_1 (relevant docs are marked with a bullet):

01. d_{123} •	06. d_9 •	11. d_{38}
02. d_{84}	07. d_{511}	12. d_{48}
03. d_{56} •	08. d_{129}	13. d_{250}
04. d_6	09. d_{187}	14. d_{113}
05. d_8	10. d_{25} •	15. d_3 •

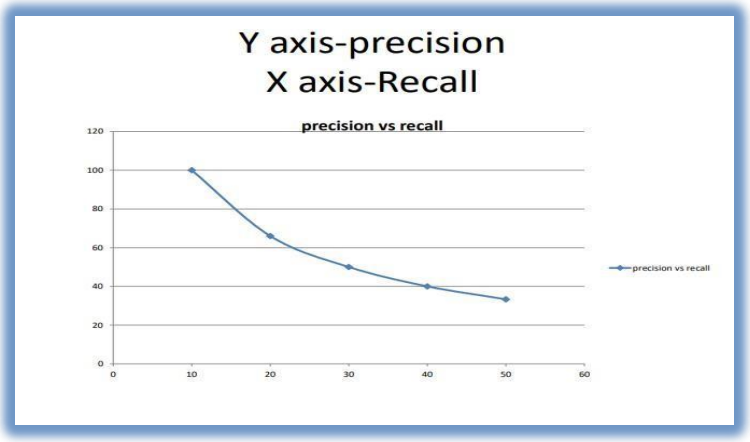
- If we examine this ranking, we observe that
  - The document d_{123}, ranked as number 1, is relevant
    - This document corresponds to 10% of all relevant documents
    - Thus, we say that we have a precision of 100% at 10% recall
  - The document d_{56}, ranked as number 3, is the next relevant
    - At this point, two documents out of three are relevant, and two of the ten relevant documents have been seen
    - Thus, we say that we have a precision of 66.6% at 20% recall

01. d_{123} •	06. d_9 •	11. d_{38}
02. d_{84}	07. d_{511}	12. d_{48}
03. d_{56} •	08. d_{129}	13. d_{250}
04. d_6	09. d_{187}	14. d_{113}
05. d_8	10. d_{25} •	15. d_3 •

R_q ={d3,d5,d9,d25,d39,d44,d56,d71,d89,d123},|R|=10

Documents	$\|R_a\|$	|A|	Precision=R_a /A	Recall= R_a /R
d123	1	1	100%	10%
D123,d84	1	2	50	10
D123,d84,**d56**	2	3	66	20
D123,d84,**d56**,d6	2	4	50	20
D123,d84,**d56**,d6,d8	2	5	40	20
D123,d84,**d56**,d6,d8,**d9**	3	6	50	30
D123,d84,**d56**,d6,d8,**d9**,d511	3	7	42.85	30
D123,d84,**d56**,d6,d8,**d9**,d511,d129	3	8	37.5	30
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187	3	9	33.33	30
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187,**d25**	4	10	40	40
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187,**d25**,d38	4	11	36.36	40
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187,**d25**,d38,d48	4	12	33	40
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187,**d25**,d38,d48,d250	4	13	30.76	40
D123,d84,**d56**,d6,d8,**d9**,d511,d129,d187,**d25**,d38,d48,d250,d	4	14	28.57	40

If we proceed with our examination of the ranking generated, we can plot a curve of precision versus recall as follows:



Thus, Precision and recall have been extensively used to evaluate the retrieval performance of IR systems or algorithms. However, a more careful reflection reveals problems with these two measures: First, the proper estimation of maximum recall for a query requires detailed knowledgeof all the documents in the collection Second, in many situations the use of a single measure couldbe more appropriate Third, recall and precision measure the effectiveness over a set of queries processed in batch mode Fourth, for systems which require a weak ordering though, recall and precision might be inadequate.

Conclusion: Implementation is concluded by executing a program to calculate precision andrecall for sample input with relevant documents Rq1 for query q1.

A. Write short answer of following questions:

1. What is precision and recall in IR systems?
2. How are recall and precision measures are defined?

B. Viva Questions:

1. What is relevance of document?
2. What are the metrics to measure information systems?
3. How are precision and recall calculated for information systems?
4. What is the problem with these two measures?

Aim : Write a program to calculate harmonic mean (F-measure) and E-measure for above example

Objective:

1. To evaluate the retrieval performance of IR systems
2. To understand importance of harmonic mean (F-measure) and E-measure in information retrieval
3. To study indexing structures for information retrieval.

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS, C/C++/.Java

Theory:

(F-Score / F-Measure)

F1 Score considers both precision and recall.

It is the harmonic mean(average) of the precision and recall.

F1 Score is best if there is some sort of balance between precision (p) & recall (r) in the system. Oppositely F1 Score isn't so high if one measure is improved at the expense of the other.

For example, if P is 1 & R is 0, F1 score is 0.

F1 Score = 2*(Recall * Precision) / (Recall + Precision)

Information Systems can be measured with two metrics: precision and recall. Thus, Precision and recall have been extensively used to evaluate the retrieval performance of IR systems or algorithms. However, a more careful reflection reveals problems with these two measures: First, the proper estimation of maximum recall for a query requires detailed knowledge of all the documents in the

collection Second, in many situations the use of a single measure could b e more appropriate Third, recall and precision measure the effectiveness over a set of queries processed in batch mode Fourth, for systems which require a weak ordering though, recall and precision might be inadequate.

Alternative measures: The harmonic mean/F measure

The F-measure is also a single measure that combines recall and precision

- The F-measure is also a single measure that combines recall and precision

$$F(j) = \frac{2}{\frac{1}{r(j)} + \frac{1}{P(j)}}$$

where

- $r(j)$ is the recall at the j-th position in the ranking
- $P(j)$ is the precision at the j-th position in the ranking
- $F(j)$ is the harmonic mean at the j-th position in the ranking

$$F(j) = \frac{2}{\frac{1}{r(j)} + \frac{1}{P(j)}}$$

Where, r (j) is the recall at the j-th position in the ranking

P (j) is the precision at the j-th position in the ranking

F (j) is the harmonic mean at the j-th position in the ranking

Determining max value of F can be interpreted as an attempt to find the best possible compromise between recall and precision. The function F assumes values in the interval [0, 1] It is 0 when no relevant documents have been retrieved and is 1 when all ranked documents are relevant Further, the harmonic mean F assumes a high value only when both recall and precision are high To maximize F requires finding the best possible compromise between recall and precision.

Alternative measures: E measure

E measure was proposed by Van Rijesbergn which combines recall and precision. User is allowed to specify whether he is more interested in recall or in precision.

E measure is defined as

$$E(j) = 1 - \frac{1 + b^2}{\frac{b^2}{r(j)} + \frac{1}{P(j)}}$$

Where,

r (j) is the recall at the j-th position in the ranking

P (j) is the precision at the j-th position in the ranking

b ≥ 0 is a user specified parameter

E (j) is the E metric at the j-th position in the ranking

If b=1 E(j) measure works as complement of the Harmonic mean F(j)

If b>1 indicates that the user is more interested in precision than in recall

If b<1 Indicates that user is more interested in recall than in precision

Notice that setting b = 1 in the formula of the E-measure yields F (j) = 1 − E (j)

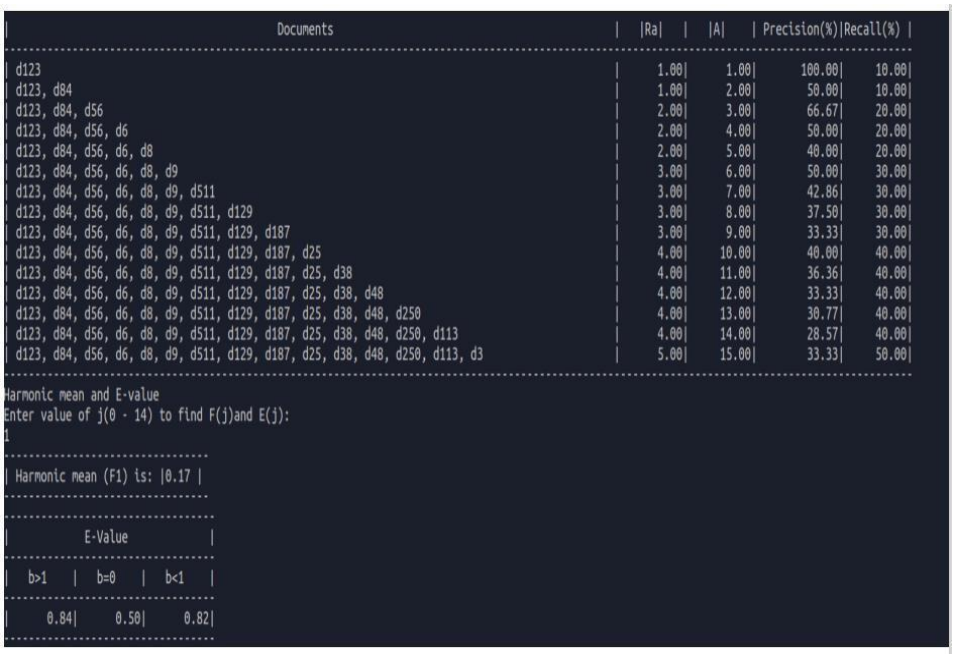
The parameter b is specified by the user and reflects the relative importance of recall and Precision.

If b = 0 E (j) = 1 − P (j) low values of b make E (j) a function of precision.

If b → ∞ lim b→∞ E (j) = 1 − r (j) high values of b make E (j) a function of recall

For b = 1, the E-measure becomes the F-measure.

Thus. Single value measures can also be stored in a table to provide a statistical summary For instance, these summary table statistics could include the number of queries used in the task the total number of documents retrieved by all queries the total number of relevant docs retrieved by all queries the total number of relevant docs for all queries, as judged by the specialists.



Documents	|Ra|	|A|	Precision(%)	Recall(%)
d123	1.00	1.00	100.00	10.00
d123, d84	1.00	2.00	50.00	10.00
d123, d84, d56	2.00	3.00	66.67	20.00
d123, d84, d56, d6	2.00	4.00	50.00	20.00
d123, d84, d56, d6, d8	2.00	5.00	40.00	20.00
d123, d84, d56, d6, d8, d9	3.00	6.00	50.00	30.00
d123, d84, d56, d6, d8, d9, d511	3.00	7.00	42.86	30.00
d123, d84, d56, d6, d8, d9, d511, d129	3.00	8.00	37.50	30.00
d123, d84, d56, d6, d8, d9, d511, d129, d187	3.00	9.00	33.33	30.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25	4.00	10.00	40.00	40.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25, d38	4.00	11.00	36.36	40.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25, d38, d48	4.00	12.00	33.33	40.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25, d38, d48, d250	4.00	13.00	30.77	40.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25, d38, d48, d250, d113	4.00	14.00	28.57	40.00
d123, d84, d56, d6, d8, d9, d511, d129, d187, d25, d38, d48, d250, d113, d3	5.00	15.00	33.33	50.00

Harmonic mean and E-value
Enter value of j(0 - 14) to find F(j)and E(j):
1

Harmonic mean (F1) is: 0.17

E-Value		
b>1	b=0	b<1
0.84	0.50	0.82

Conclusion: Implementation is concluded by executing a program to calculate (F-measure) and E-measure for sample input used in above example.

A. Write short answer of following questions:

1. What is harmonic mean (F-measure) and E-measure in IR systems?
2. How are (F-measure) and E-measure calculated?

B. Viva Questions:

1. What is difference between (F-measure) and E-measure?
2. What are the metrics to measure information systems?
3. What is the advantage of (F-measure) and E-measure?

6)Aim : To implement a program for feature extraction in 2D color images (any features like shape, texture , size, owner, type of file etc.

Objective:

To study features extraction process of 2D color images for information retrieval

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS, C/C++/.Java

Input:

Image file

Output:

Features of Image file

Theory:

Introduction:

Technology determines the types and amounts of information we can access. Currently, a large fraction of information originates in silicon. Cheap, fast chips and smart algorithms are helping digital data processing take over all sorts of information processing. Consequently, the volume of digital data surrounding us increases continuously. However, an information-centric society has additional requirements besides the availability and capability to process digital data. We should also be able to find the pieces of information relevant to a particular problem. Having the answer to a question but not being able to find it is equivalent to not having it at all. The increased volume of information and the wide variety of data types make finding information a challenging task. Current searching methods and algorithms are based on assumptions about technology and goals that seemed reasonable before the widespread use of computers. However, these assumptions no longer hold in the context of information retrieval systems. The pattern originated in the information retrieval domain.

However, information retrieval has expanded into other fields like office automation, genome

databases, fingerprint identification, medical imaging, data mining, multimedia, etc. Since the pattern works with any kind of data, it is applicable in many other domains. You will see examples from text searching, telecommunications, stock prices, medical imaging and trademark symbols. The key idea of the pattern is to map from a large, complex problem space into a small, simple feature space. The mapping represents the creative part of the solution. Every type of application uses a different kind mapping. Mapping into the feature space is also the hard part of this pattern. Traditional searching algorithms are not viable for problems typical to the information retrieval domain. Since they were designed for exact matching, their use for similarity search is cumbersome. In contrast, feature extraction provides an elegant and efficient alternative. With information retrieval expanding into other fields, this pattern is applicable in a wide range of applications. Work with an alternative, simpler representation of data. The representation contains some information that is unique to each data item. This computation is actually a function. It maps from the problem space into a feature space. For this reason, it is also called feature extraction process.

Feature Extraction**:**

Texture is an important feature that identifies the object present in any image. The texture is defined by the spatial distribution of pixels in the neighborhood of an image. The gray level spatial dependency is represented by a two-dimensional matrix known as GLCM and it is used for texture analysis. The GLCM matrix specifies that how often the pairs of pixels with certain values occur in an image. The statistical measures are then derived using the GLCM matrix. The textural features representthe spatial distribution of gray tonal variations within a specified area. In images, the neighboring pixelis correlated and spatial values are obtained by the redundancy between the neighboring pixel values.The color features are represented by color histograms in six color spaces namely RGB, HSV, LAB,CIE, HUE and OPP.

The textural features are considered for classifying the image. These textural features are calculated in the spatial domain and a set of gray tone spatial dependency matrix was computed. The textural features are computed using GLCM matrix in four different orientation angles. The textural features are based onthe fact that describes how the gray tone appears in a spatial relationship to another.

GRAY LEVEL CO-OCCURENCE MATRIX (GLCM)

In statistical texture analysis, from the distribution of intensities the texture features are obtained at specified position relative to one another in an image. The statistics of texture are classified into first order, second order and higher order statistics. The method of extracting second order statistical texture features is done using Gray Level Co-occurrence Matrix (GLCM). First order texture measure is not related to pixel neighbor relationships and it is calculated from the original image. GLCM considers the relation

between two pixels at a time, called reference pixel and a neighbor pixel. A GLCM is definedby a matrix in which the number of rows and columns are equal to the number of gray levels G in an image. The matrix element P (I, j | Ax, By) is the relative frequency where I and j represent the intensity and bothare separated by a pixel distance Ax, By. The different textural features such as energy, entropy, contrast,homogeneity, correlation, dissimilarity, inverse difference moment and maximum probability can be computed using GLCM matrix.

Significance of Extracted Feature**:**

1. Color: It signifies the object identification and extraction of from scene.
2. Brightness: Brightness is one of the most significant pixel characteristics. Brightness should beused only for no quantitative references to physiological sensations and perceptions of light.
3. Entropy: It characterized the texture in image
4. Contrast: Contrast is the dissimilarity or difference between things.
5. Shape of image
6. Size of image
7. Owner, file name, file type etc.

Algorithm
1. Open colored 2D bitmap file in binary mode. 2. Read the header structure 3. Extract the various feature 4. Print the values of features

Conclusion**:**

Implementation is concluded by stating the fundamentals of feature extraction from image file.

Write short answer of following questions :

1. How images are indexed
2. Explain how color is extracted from image
3. What is multimedia IR?Discuss steps on which data retrieval relies

Viva Questions:

1. What is use of image features.
2. Enlist some of the features of image and its applications.
3. How to compare two images and calculate the relevancy

Aim :7) Build the web crawler to pull product information and links from an e-commerce website. (Python)

Objective: -

To understand the working of web crawler and implement it

Outcomes:

At the end of the assignment the students should have

1. Understood how web crawler works

PEOs, POs, PSOs and COs satisfied

POs: a,b,c,d,e **PEOs: I,III** **PSOs: 1,2,3** **COs: 1,2,3,4,5,6,7**

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS/C/C++/Java

Theory:

Search Engines

A program that searches documents for specified keywords and returns a list of the documents where the keywords were found is a search engine. Although search engine is really a general class of programs, the term is often used to specifically describe systems like Google, Alta Vista and Excite that enable users to search for documents on the World Wide Web and USENET newsgroups.

Typically, a search engine works by sending out a spider to fetch as many documents as possible. Another program, called an indexer, then reads these documents and creates an index based on the words contained in each document. Each search engine uses a proprietary algorithm to create its indices such that, ideally, only meaningful results are returned for each query. Search engines are special sites on the Web that are designed to help people find information stored on other sites.

There are differences in the ways. Various search engines work, but they all perform three basic tasks:

- They search the Internet - based on important words.
- They keep an index of the words they find, and where they find them.
- They allow users to look for words or combinations of words found in that index.

Fig.1 shows general search engine architecture. Every engine relies on a *crawler module* to

provide the grist for its operation. Crawlers are small programs that browse the Web on the search engine's behalf, similar to how a human user would follow links to reach different pages. The programs are given a starting set of URLs, whose pages they retrieve from the Web.

The crawlers extract URLs appearing in the retrieved pages, and give this information to the crawler *control* module. This module determines what links to visit next, and feeds the linksto visit back to the crawlers. (Some of the functionality of the crawler control module may be implemented by the crawlers themselves.) The crawlers also pass the retrieved pages into a *page repository*. Crawlers continue visiting the Web, until local resources, such as storages, are exhausted.

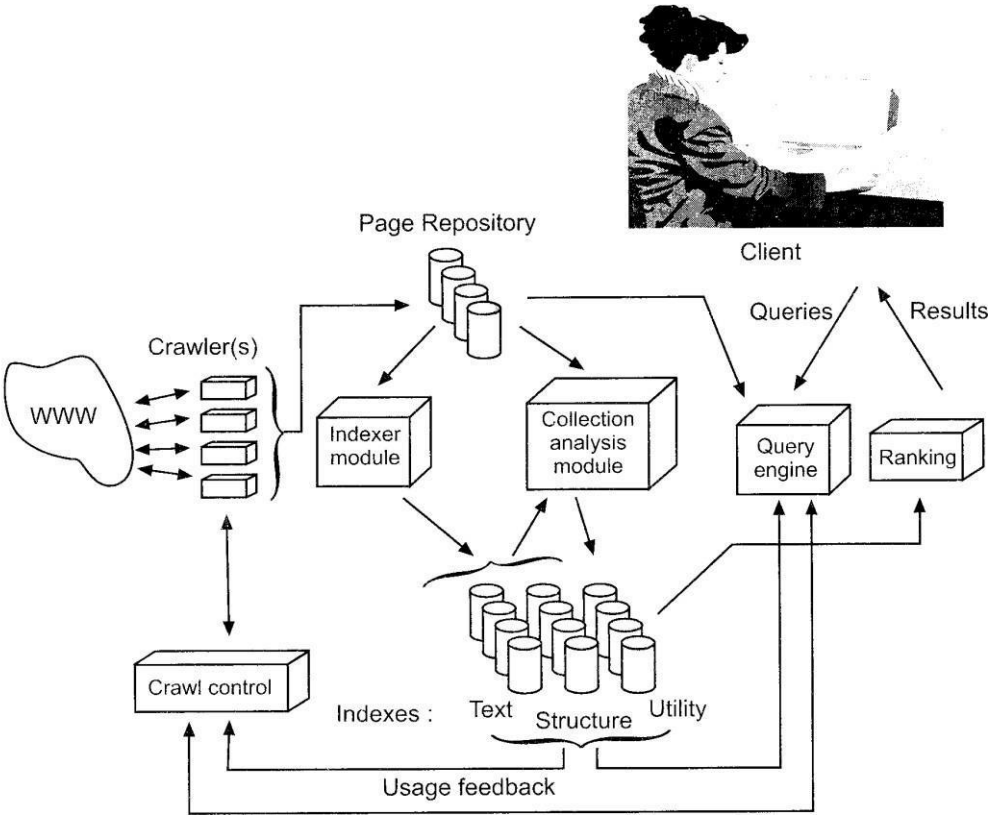


Fig: General search engine architecture

Web Crawlers

Web crawlers are programs that exploit the graph structure of the Web to move from page to page. It may be observed that the noun 'crawler' is not indicative of the speed of these programs, as they can be considerably fast. A key motivation for designing Web crawlers has beento retrieve Web pages and add them or their representations to a local repository. Such a repositorymay then

serve particular application needs such as those of a Web search engine. In its simplest form, a crawler starts from a *seed* page and then uses the external links within it to attend to other pages. The Crawler is the means by which WebCrawler collects pages from the Web. It operates by iteratively downloading a web page, processing it, and following the links in that page to other Web pages, perhaps on other servers. The end result of crawling is a collection of Web pages, HTML or plain text at a central location. The collection policy implemented in the crawler determines what pages are collected, and which of those pages are indexed. Although at first glance the process of crawling appears simple, many complications occur in

practice. In a more traditional IR system, the documents to be indexed are available locally in a database or file system. WebCrawler's first information retrieval system was based on Salton's vector- space retrieval model.

The first system used a simple vector-space retrieval model. In the vector- space model, the queries and documents represent vectors in a highly dimensional word space. The component of the vector in a particular dimension is the significance of the word to the document. For example, if a particular word is very significant to a document, the component ofthe vector along that word's axis would be strong. In this vector space, then, the task of querying becomes that of determining what document vectors are most similar to the query vector. Practically speaking, this task amounts to comparing the query vector, component by component, to all the document vectors that have a word in common with the query vector. WebCrawler determined a similarity number for each of these comparisons that formed the basis of the relevance score returned to the user. WebCrawler's first IR system had three pieces: a query processing module, an inverted full-text index, and a metadata store. The query processing module parses the searcher's query, looks up the words in the inverted index, forms the result list, looks up the metadata for each result, and builds the HTML for the result page. The query processing module used a series of data structures and algorithms to generate results for a givenquery. First, this module put the query in a canonical form, and parsed each space-separated wordin the query. If necessary, each word was converted to its singular form using a modified Porter stemming algorithm and all words were filtered through the stop list to obtain the final list of words. Finally, the query processor looked up each word in the dictionary, and ordered the list of words for optimal query execution. WebCrawler's key contribution to distributed systems is to show that a reliable, scalable, and responsive system can be built using simple techniques for handling distribution, load balancing, and fault tolerance.

Robot Exclusion

The Robot Exclusion Standard, also known as the Robots Exclusion Protocol or robots.txt protocol, is a convention to prevent co-operating web crawlers and other web robots from accessing all or a part of a website which is otherwise publicly viewable. Robots are often used by search engines to categorize and archive web sites, or by webmasters to proofread source code. The standard is different but can be used in conjunction with sitemaps, a robot *inclusion* standard for websites.

A robots.txt file on a website will function as a request that specified robots ignore specified files or directories in their search. This might be, for example, out of preference for privacy from search engine results, or the belief that the content of the selected directories might be misleading or irrelevant to the categorization of the site as a whole, or out of desire that an application only operates on certain data. A person may not want certain pages indexed. Crawlers should obey the Robot Exclusion Protocol.

The Robots Exclusion Protocol (REP) is a very simple but powerful mechanism available to webmasters and SEOs alike. Perhaps it is the simplicity of the file that means it is often overlooked and often the cause of one or more critical SE0 issues. To this end, we have attempted to pull together tricks, tips and examples to assist with the implementation and management of your robots.txt file. As many of the non-standard REP declarations supported by Google, Yahoo and Bing may change, we will be providing updates to this in the future.

The robots.txt file defines the Robots Exclusion Protocol (REP) for a website. The file defines directives that exclude Web robots from directories or files per website host. The robots.txt file defines crawling directives, not indexing directives. Good Web robots adhere to directives in your robots.txt file. Bad Web robots may not. Do not rely on the robots.txt file to protect private or sensitive data from search engines. The robots.txt file is publicly accessible and so do not include any files or folders that may include business critical information.

For example: Website analytics folders (/web stats/, /stats/ etc.)

Test or development areas (/test/, /dev/)

XML Sitemap element if your URL structure contains vital taxonomy.

If a URL redirects to a URL that is blocked by a robots.txt file, the first URL will be reported as being blocked by robots.txt in Google Webmaster Tools. Search engines may cache your robots.txt file (For example: Google may cache your robots.txt file for 24 hours). When deploying a new website from a development environment always check the robots.txt file to ensure no key directories are excluded. Excluding files using robots.txt may not save the crawl budget from the same crawl session. For example: if Google cannot access a number of files it may not crawl other files in their place. URLs excluded by REP (Robots Exclusion Protocol) may still appear in a search engine index.

Program Implementation: Code written in Java to implement of the same with appropriate output.

Algorithm

1. Make User Interface
2. Input the URL of any website
3. Establish HTTP connection
4. Read HTML page source code
5. Extract Hyperlinks of HTML page
6. Display the list of hyperlinks on the same page

Conclusion: Implementation is concluded by stating the basic working of web crawler.

Write short answer of following questions:

1. What are the crawler architectures.
2. Explain Harvest architecture
3. Explain the working of GOOGLE crawler
4. Explain Challenges involved in searching web.
5. Explain Meta searches with examples

Viva Questions:

1. What is robot.txt?
2. What is the significance of robot.txt?
3. What are the strategies used by crawler.
5. What is page rank?
6. What is significance of dampening factor?

Aim : Write a program to find the live weather report (temperature, wind speed, description, and weather) of a given city. (Python).

Objective: -

1. To Get Weather Information using Python
2. To evaluate the performance of the IR system and understand user interfaces for searching.
3. To understand information sharing on the web

Infrastructure: Desktop/ laptop system with Linux or its derivatives.

Software used: LINUX/ Windows OS/ Virtual Machine/ IOS/C/C++/Java

Theory:

Weather Information using Python

Python is a growing language that has become increasingly popular in the programming community. One of Python's key features is that it's easy to work with APIs on websites and many weather entities have their API which you can access with just a couple of lines of code. One such great API is the Open Weather Map's API, with it you can build a small program to access any given location's weather forecast anywhere across the globe! This article will help you to get weather information using Python.

What is OpenWeatherMap?

The OpenWeatherMap (OWM) is a helpful and free way to gather and display weather information. Because it's an open-source project, it's also free to use and modify in any way. OWM offers a variety of features and is very flexible. Because of these qualities, it offers a lot of benefits to developers. One of the major benefits of OWM is that it's ready to go. Unlike

other weather applications, OWM has a web API that's ready to use. You don't have to install any software or set up a database to get it up and running. This is a great option for developers who want to get weather reading on a website quickly and efficiently.

It has an API that supports HTML, XML, and JSON endpoints. Current weather information extended forecasts, and graphical maps can be requested by users. These maps show cloud cover, wind speed as well as pressure, and precipitation.

Code Snippets

Sample Solution:

Python Code:

```
import requests
from pprint import pprint
def weather_data(query):
    res=requests.get('http://api.openweathermap.org/data/2.5/weather?'+query+'&APPID=**************************&units=metric');
    return res.json();
def print_weather(result,city):
    print("{}'s temperature: {}°C ".format(city,result['main']['temp']))
    print("Wind speed: {} m/s".format(result['wind']['speed']))
    print("Description: {}".format(result['weather'][0]['description']))
    print("Weather: {}".format(result['weather'][0]['main']))
def main():
    city=input('Enter the city:')
    print()
    try:
      query='q='+city;
      w_data=weather_data(query);
      print_weather(w_data, city)
      print()
    except:
      print('City name not found...')
if __name__=='__main__':
    main()

```

Sample Output:

```
Enter the city: Brazil
Brazil's temperature: 16.45°C
Wind speed: 2.1 m/s
Description: clear sky
Weather: Clear
```

Conclusion: Thus, we have successfully implemented a program to find the live weather report (temperature, wind speed, description, and weather) of a given city using Python.

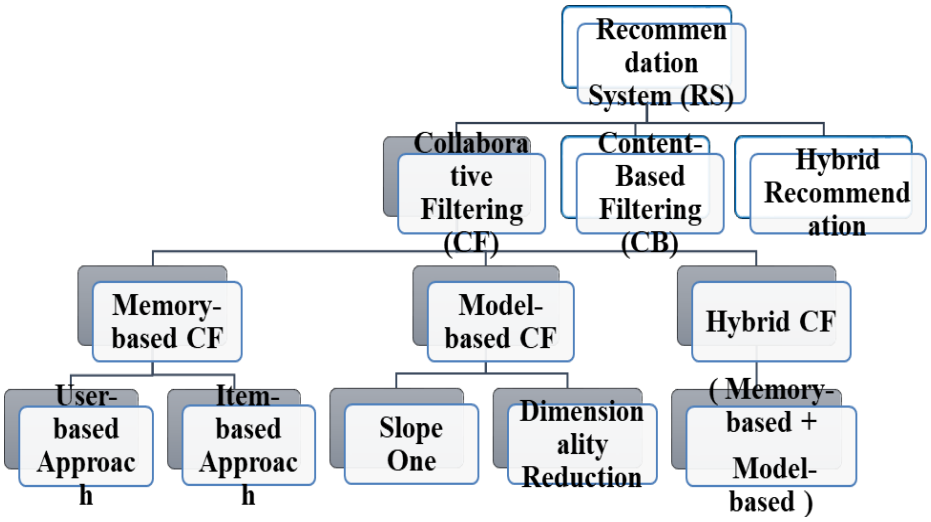


Fig: Recommendation system block diagram

Conclusion: **Thus, we have studied collaborative recommender system**

A. Write short answer of following questions:

1. Explain Collaborative Filtering Recommendation of Documents and products.
2. Explain Content Based Filtering Recommendation of Documents and products.

B. Viva Questions:

1. What are recommendation systems?
2. How are recommendation systems classified?